

PROGRAMACIÓN AVANZADA

Interfaces.



Ing. Matias Macrino

INTERFACES

▶ Definición de interfaz:

- ▶ Una interfaz es un **conjunto de métodos abstractos**.
- ▶ La forma más fácil de entender una interfaz es pensarla como una clase abstracta pura, es decir, una **clase cuyos servicios son todos abstractos**.
- ▶ Es un convenio (contrato) con la clase que implementa la interfaz.
- ▶ Una interfaz **no establece lo que un método tiene que hacer y como hacerlo**, sino el formato (nombre, parámetros y tipo de devolución) que este debe tener.

INTERFACES

▶ Consideraciones al crear una interfaz:

- ▶ **Todos los métodos definidos en una interfaz son públicos y abstractos.**
- ▶ **En una interfaz es posible definir constantes.** Además de métodos, las interfaces pueden contener constantes, las cuales son implícitamente, públicas y estáticas.
- ▶ **Una interfaz no es una clase.** No pueden contener métodos con código, constructores o variables. No es posible crear objetos de una interfaz.

INTERFACES

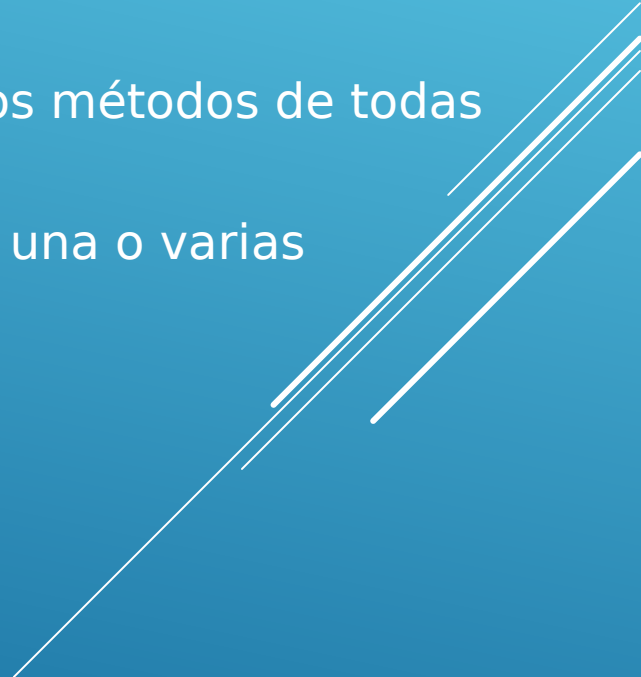
► Implementación de una interfaz:

- El objetivo de las interfaces es proporcionar un formato común de métodos para las clases. Para forzar a que una clase defina el código para los métodos declarados en una determinada interfaz, la clase deberá **implementar** la interfaz.
- En la definición de una clase, se utiliza la palabra **implements** para indicar que interfaz se ha de implementar.

Ejemplo: `public class MiClase implements MiInterface {`
 `.....`
 `}`

INTERFACES

► Implementación de una interfaz:

- Es obligatorio definir el código (implementar) de todos los métodos existentes de la interfaz.
 - Una clase puede implementar mas de una interfaz. Implementado los métodos de todas las interfaces.
 - Una clase puede heredar otra clase e implementar al mismo tiempo una o varias interfaces.
 - Una interfaz puede “heredar” otras interfaces.
- 
- Several white lines of varying lengths and orientations are positioned in the bottom right corner of the slide, creating a modern, abstract graphic element.

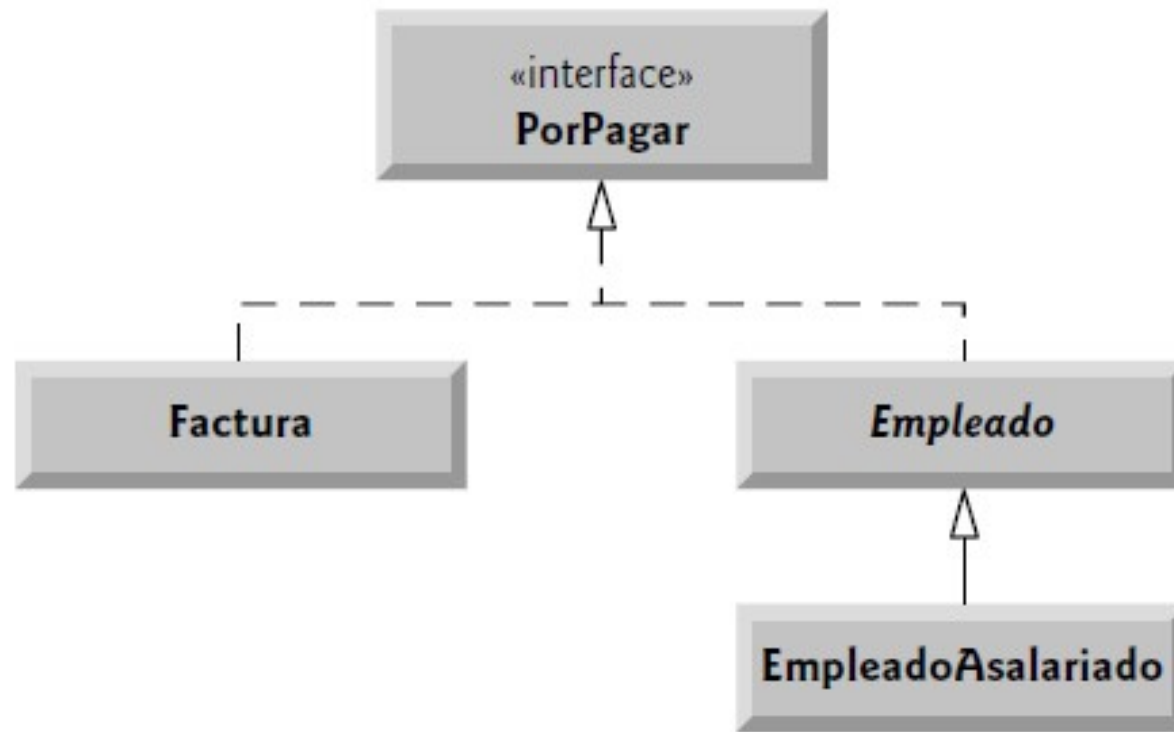
INTERFACES

► Ejemplo Practico:

- Nos piden desarrollar una aplicación que pueda determinar los pagos para “**empleados**” que se dividen en “**empleados a salarios**” con su pago mensual y “**facturas**” por igual.
- De las facturas nos piden almacenar:
 - Numero de pieza.
 - Descripción de pieza.
 - Cantidad.
 - Precio por articulo.
- De los empleados nos piden almacenar:
 - Nombre y Apellido.
 - Seguro social.
 - Legajo.

INTERFACES

► Ejemplo Practico:



INTERFACES

► Ejemplo Practico:

Generamos la interface con el método que deseamos que sea común a nuestros objetos.

```
public interface PorPagar {  
    double obtenerMontoPago();  
}
```


INTERFACES

► Ejemplo Practico:

```
public class Factura implements PorPagar{  
  
    private String numeroPieza;  
    private String descripcionPieza;  
    private int cantidad;  
    private double precioPorArticulo;  
  
    public Factura() {
```

En nuestro objeto
implementamos la interface

Sobrescribimos el método de nuestra
interface

```
        @Override  
        public double obtenerMontoPago() {  
            return this.getCantidad() * this.getPrecioPorArticulo();  
        }  
    }
```

INTERFACES

► Ejemplo Practico:

```
public abstract class Empleado implements PorPagar{  
  
    private String nombreApellido;  
    private String seguroSocial;  
    private String legajo;  
  
    public Empleado() {  
          
    }  
}
```

Generamos una clase abstracta para que no nos obligue a sobrescribir el método de la interface.

INTERFACES

► Ejemplo Practico:

```
public class EmpleadoASalariado extends Empleado {  
  
    private double pagoMensual;  
  
    public EmpleadoASalariado() {  
        super();  
    }  
}
```

Creamos nuestra clase que hereda de empleado

Sobrescribimos el método de nuestra interface

```
@Override  
public double obtenerMontoPago() {  
    return this.getPagoMensual();  
}
```

INTERFACES

► Ejemplo Practico:

```
public static void main(String[] args) {  
  
    PorPagar[] objetosPorPagar = new PorPagar[4];  
  
    objetosPorPagar[0] = new Factura("000001122144", "Asiento", 2, 375.00);  
    objetosPorPagar[1] = new Factura("012344123319", "Llanta", 4, 750.75);  
    objetosPorPagar[2] = new EmpleadoASalariado("John Smith", "111-2331-111", "L00085", 45000.80);  
    objetosPorPagar[3] = new EmpleadoASalariado("Lisa Barnes", "112-2311-011", "L00159", 75000.05);  
  
    for(int i = 0; i < objetosPorPagar.length; i++){  
        System.out.println(objetosPorPagar[i].toString() + "\tPago Vencido: "  
                            + objetosPorPagar[i].obtenerMontoPago() );  
    }  
}
```

INTERFACES

► Ejemplo Practico:

```
Factura [numeroPieza=000001122144, descripcionPieza=Asiento, cantidad=2, precioPorArticulo=375.0]    Pago Vencido: 750.0
Factura [numeroPieza=012344123319, descripcionPieza=Llanta, cantidad=4, precioPorArticulo=750.75]    Pago Vencido: 3003.0
Empleado [nombreApellido=John Smith, seguroSocial=111-2331-111, legajo=L00085]    Pago Vencido: 45000.8
Empleado [nombreApellido=Lisa Barnes, seguroSocial=112-2311-011, legajo=L00159]    Pago Vencido: 75000.05
```

INTERFACES EN JAVA 8

► Interfaces Aumentadas:

- A partir de su versión 8, en Java se pueden declarar **métodos no abstractos, es decir con implementaciones**, gracias a dos mejores:

- **Métodos por default (default methods)**

- **Métodos estáticos (static methods)**

- **Clases Anonimas (Anonymous Classes)**

INTERFACES EN JAVA 8

- ▶ Interfaces Aumentadas:
- ▶ **Clases Anónimas (Anonymous Classes):** En Java 8 las interfaces pueden implementarse a partir de las denominadas clases anónimas.
- ▶ Estas mismas, no requieren que se genere una nueva clase y utilizar la palabra reservada de ***implements***.
- ▶ Su desventaja es que no pueden ser utilizadas desde otras clases, el alcance de su declaración es dentro del contexto que se la utilice.

INTERFACES EN JAVA 8

- ▶ Interfaces Aumentadas:
- ▶ **Métodos por default (default methods):** En Java 8 las interfaces puede contener métodos **default public** con implementaciones predeterminadas concretas.
- ▶ Estos métodos, especifican **el como** deben realizarse las operaciones cuando la clase implementadora no sobrescriban los métodos.
- ▶ Para su declaración de debe de colocar la palabra clave **default** antes del tipo de valor de retorno del método.

INTERFACES EN JAVA 8

- ▶ Interfaces Aumentadas:

- ▶ **Métodos por default (default methods):**

- ▶ Las clases implementadoras no tienen que sobrescribir los métodos **default** de la interfaz, pero pueden hacerlo en caso de ser necesario.
- ▶ **Los métodos default en Java 8 permiten evolucionar las interfaces existentes al agregar nuevos métodos a esas interfaces sin descomponer el código que la usa.**

INTERFACES EN JAVA 8

► Interfaces Aumentadas:

- **Métodos estáticos (*static methods*):** Antes de Java 8, era común asociar a una interfaz a una clase que tuviera métodos ayudantes ***static*** para trabajar con objetos que implementan la interfaz.
- Con estos métodos (static) de una interfaz, dichos métodos ayudantes pueden ahora declararse directamente en las interfaces, en vez de hacerlo en cada una de las clases.

INTERFACES

► Ejercitacion:

- Generar un programa en el cual se pueda obtener el área de las siguientes figuras geometricas:
 - Circulo ($\pi * \text{radio}^2$)
 - Rectángulo ($\text{base} * \text{altura}$)
 - Triangulo ($\text{base} * \text{altura} / 2$)
 - Cuadrado (lado^2).
- Implementar un objeto por cada figura y utilizar interface (métodos abstractos y constantes) para el desarrollo del ejercicio.

PROGRAMACIÓN AVANZADA

Clases Internas



Ing. Matias Macrino

CLASES INTERNAS

- ▮ Clases Internas o Anidadas:

- ▮ No suelen representar un mecanismo demasiado útil y práctico para el desarrollo de aplicaciones habituales.
- ▮ Este concepto se entiende como una clase que contiene otra en su interior. Pueden presentarse las siguientes situaciones:
 - ▮ Clase internas estándares
 - ▮ Clase internas locales a métodos.
 - ▮ Clases internas estáticas.

CLASES INTERNAS

- ▮ Clases Internas estándares:
- ▮ Se definen como una clase no estática definida en el interior de otra.
- ▮ Para generar una instancia de la clase interna, es necesario previamente una instancia de la clase externa.
Esto no aplica, si deseamos crear una instancia dentro de la clase externa.
- ▮ Una clase interna tiene acceso al resto de los miembros de la clase externa.
- ▮ Al ser un miembro mas de la clase podemos usar modificares para su acceso, tales como ***private, static, protected, abstrac y public.***

CLASES INTERNAS

```
public class Externa {  
  
    /*  
        DECLARACION DE LA CLASE INTERNA  
    */  
    public class Interna {  
  
        public void mensajeInterna() {  
            Externa externa = new Externa(); //INSTANCIA DE LA CLASE EXTERNA  
            System.out.println("Soy un metodo de mi CLASE INTERNA. Class: " + this.getClass().getName());  
            System.out.println("Estoy llamando al metodo de mi CLASE EXTERNA");  
            externa.mensajeExterna(externa);  
        }  
    }  
  
    public void mensajeExterna() {  
        System.out.println("Soy un metodo de mi CLASE EXTERNA. Class: " + this.getClass().getName());  
    }  
  
    public void mensajeExterna(Object o) {  
        System.out.println("Soy un metodo de mi CLASE EXTERNA. Class: " + o.getClass().getName());  
    }  
}
```

```
public class PrincipiaplInternaEstandar {  
  
    public static void main(String[] args) {  
        Externa externa = new Externa();  
        externa.mensajeExterna();  
        //INSTANCIA DE LA CLASE INTERNA A TRAVES DE LA CLASE EXTERNA  
        Externa.Interna interna = externa.new Interna();  
        interna.mensajeInterna();  
    }  
}
```

```
"C:\Program Files\Java\jdk1.8.0_171\bin\java.exe" ...  
Soy un metodo de mi CLASE EXTERNA. Class: edu.usal.clasetres.Externa  
Soy un metodo de mi CLASE INTERNA. Class: edu.usal.clasetres.Externa$Interna  
Estoy llamando al metodo de mi CLASE EXTERNA  
Soy un metodo de mi CLASE EXTERNA. Class: edu.usal.clasetres.Externa
```

CLASES INTERNAS

- ▮ Clases Internas locales a método:
- ▮ Su definición se encuentra dentro de un método de la clase externa.
- ▮ Como su contexto, es el método al que pertenece ***solamente podrá ser instanciada en el interior de dicho método.***
- ▮ Hay que tener en cuenta que ***una clase local a método no puede acceder a las variables locales definidas en dicho método,*** salvo que sean constantes.

CLASES INTERNAS

▮ Clases Internas locales a método:

```
public class ExternaMetodo {  
  
    void proceso() {  
        int numero = 0;  
        final double decimal = 12.5;  
        class InternaMetodo {  
            void muestra() {  
                System.out.println("Clase local metodo. " +  
                                   "\nDecimal: " + decimal);  
            }  
        }  
        InternaMetodo internaMetodo = new InternaMetodo();  
        internaMetodo.muestra();  
    }  
}
```

```
public class PrincipalInternaMetodo {  
  
    public static void main(String[] args) {  
        ExternaMetodo externaMetodo = new ExternaMetodo();  
        externaMetodo.proceso();  
    }  
}
```

```
C:\Program Files\Java\jdk1.6.0_171\bin  
Clase local metodo.  
Decimal: 12.5
```

CLASES INTERNAS

- ▮ Clases Internas estáticas:
- ▮ La clase interna resulta ser un miembro de acceso estático.
- ▮ ***Estas clases no pueden acceder a las variables y métodos no estáticos de la clase externa.***

CLASES INTERNAS

▮ Clases Internas locales a métodos

```
public class ClaseEstatica {  
  
    static class InternaEstatica {  
  
        public void mensajeEstatica() {  
            System.out.println("Mensaje desde la clase interna estatica");  
        }  
    }  
  
}
```

```
public class PrincipalInternaEstatica {  
  
    public static void main(String[] args) {  
  
        ClaseEstatica.InternaEstatica internaEstatica = new ClaseEstatica.InternaEstatica();  
        internaEstatica.mensajeEstatica();  
  
    }  
  
}
```

```
"C:\Program Files\Java\jdk1.8.0_171\bin\java.exe" ...  
Mensaje desde la clase interna estatica
```

PROGRAMACIÓN AVANZADA

Uso de properties



Ing. Matias Macrino

PROPERTIES

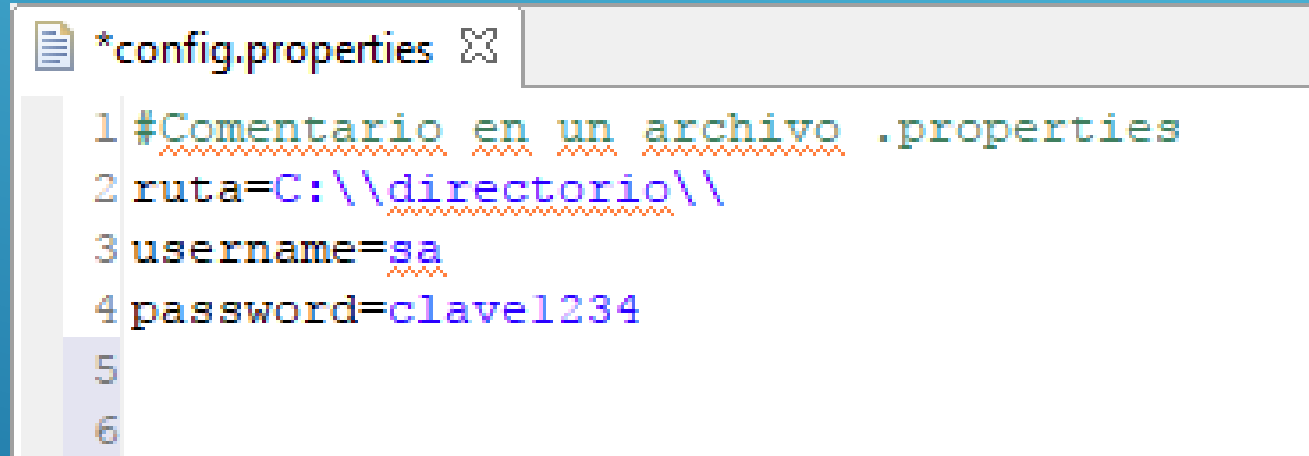
▮ ***Clase Properties:***

- ▮ Java nos proporciona el uso de archivos de propiedades, en donde podemos guardar algunos datos útiles, como configuración de parámetros.
- ▮ Un archivo de propiedades, es nada menos que un archivo de texto plano con extensión “.properties”.
- ▮ Cada línea del archivo cuenta con una pareja de datos del tipo clave=valor

PROPERTIES

- ▮ ***Clase Properties:***

- ▮ Ejemplo de archivo properties:



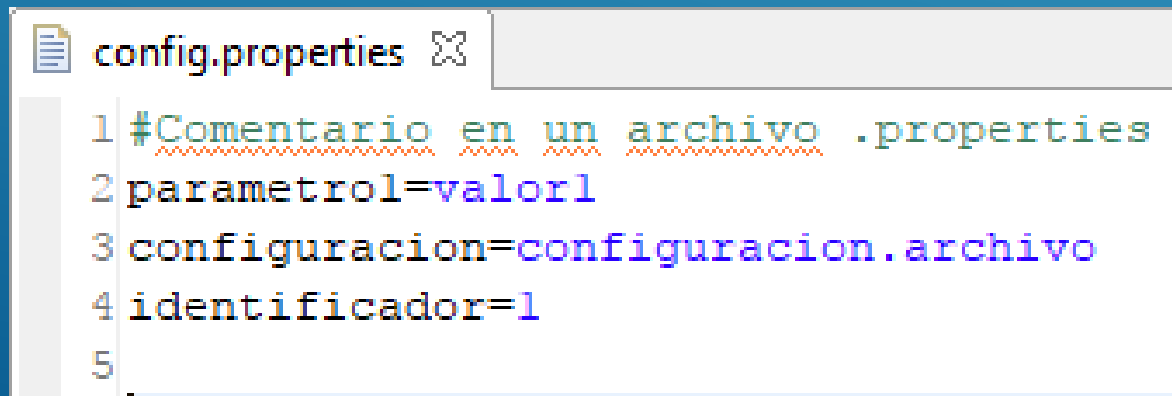
```
*config.properties X
1 #Comentario en un archivo .properties
2 ruta=C:\\directorio\\
3 username=sa
4 password=clave1234
5
6
```

- ▮ Cada clave representa un parámetro de configuración y el valor es el valor que tomara dicho parámetro.

PROPERTIES

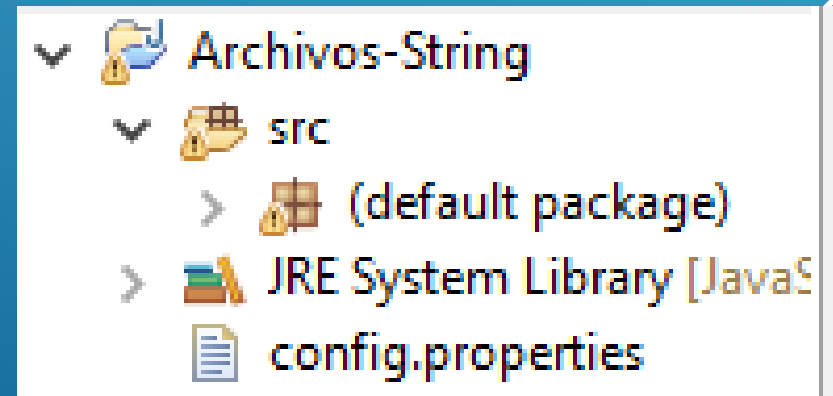
▮ *Clase Properties - Lectura:*

- ▮ Java nos brinda una manera sencilla de leer estos archivos con extensión “.properties”.
- ▮ Supongamos que tenemos el archivo “config.properties”, dentro de nuestro directorio raíz del proyecto.



The screenshot shows a code editor window titled "config.properties" with a search icon. The code inside is as follows:

```
1 #Comentario en un archivo .properties
2 parametrol=valor1
3 configuracion=configuracion.archivo
4 identificador=1
5
```



The screenshot shows a project explorer view with the following structure:

- Archivos-String
 - src
 - (default package)
 - JRE System Library [JavaS
 - config.properties

PROPERTIES

▮ **Clase *Properties* - Lectura:**

- ▮ La clase “***Properties***”, que pertenece a ***java.útil***, es la que nos permite la lectura del archivo “.properties” y después acceder a sus parámetros.
- ▮ Primero debemos de instanciar un objeto de la clase ***Properties***.
- ▮ Segundo utilizamos el método ***load(InputStream input)*** o ***load(Reader input)***, lo que nos permite este método es cargar el archivo.
- ▮ Tercero para acceder a las parámetros de nuestro archivo utilizamos el método ***getProperty(String Key)***, que pasándole la Key obtendremos el valor del parámetro deseado.

PROPERTIES

▮ *Clase Properties - Lectura:*

```
1+ import java.io.FileNotFoundException;
5
6 public class UsoProperties {
7
8-     public static void main(String[] args) throws FileNotFoundException, IOException {
9         Properties properties = new Properties();
10        properties.load(new FileReader("config.properties"));
11        System.out.println(properties.getProperty("parametro1"));
12        System.out.println(properties.getProperty("configuracion"));
13        System.out.println(properties.getProperty("identificador"));
14    }
15
16 }
17
```

Console Problems Javadoc Declaration Call Hierarchy

<terminated> UsoProperties [Java Application] C:\Program Files (x86)\Java\jre1.8.0_151\bin\javaw.exe (24 de mar. de 2018 20:46:39)

```
valor1
configuracion.archivo
1
```

PROPERTIES

▮ Clase Properties - Leer todos:

- ▮ La clase **Properties** nos ofrece una manera de obtener todas la claves de nuestro archivo y por ultimo acceder al valor de dichas claves. Esto lo

```
1+ import java.io.FileNotFoundException;
6
7 public class UsoProperties {
8
9     public static void main(String[] args) throws FileNotFoundException, IOException {
10         Properties properties = new Properties();
11         properties.load(new FileReader("config.properties"));
12         Enumeration<Object> enumeration = properties.keys();
13         while(enumeration.hasMoreElements()) {
14             System.out.println(properties.getProperty(String.valueOf(enumeration.nextElement())));
15         }
16     }
17 }
18
```

<terminated> UsoProperties [Java Application] C:\Program Files (x86)\Java\jre1.8.0_151\bin\javaw.exe (24 de mar. de 2018 20:52:58)

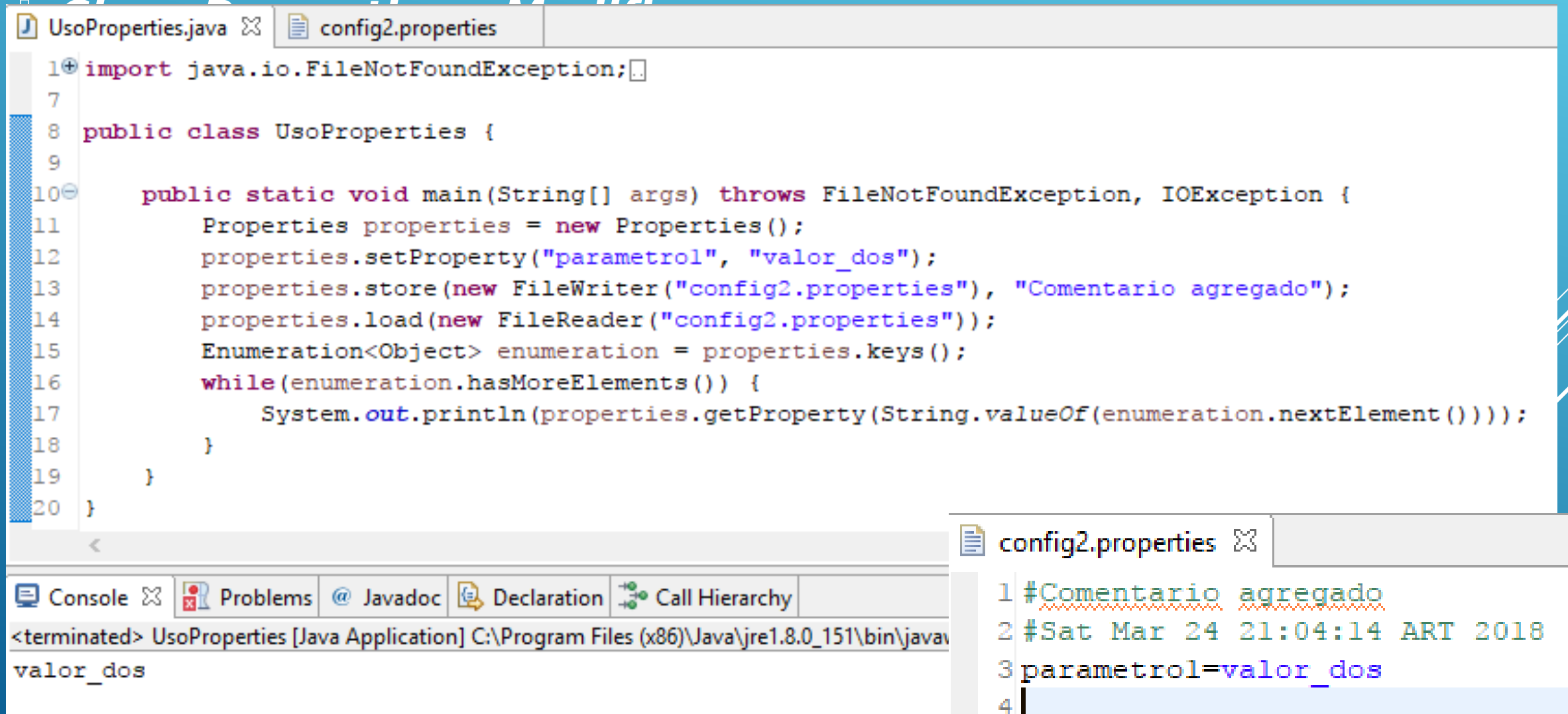
valor1
1
configuracion.archivo

PROPERTIES

▮ *Clase Properties - Modificar:*

- ▮ La manera de modificar un parámetro de nuestro archivo es mediante el método ***setProperty(String key, String value)***. Además de modificar podemos agregar nuevos parámetros con dicho método.
- ▮ Para que estos cambios sean guardados dentro de nuestro archivo debemos de utilizar el método ***store(OutputStream output)*** o ***store(Writer output)***
- ▮ Este método acepta un segundo argumento String que será un comentario que se agregara a la cabecera de nuestro archivo.

PROPERTIES



The screenshot shows an IDE with two tabs: 'UsoProperties.java' and 'config2.properties'. The Java file contains a class 'UsoProperties' with a 'main' method that reads properties from 'config2.properties' and prints them. The console output shows the program's execution, including the file path and the printed properties.

```
UsoProperties.java  config2.properties

1+ import java.io.FileNotFoundException;
7
8 public class UsoProperties {
9
10+     public static void main(String[] args) throws FileNotFoundException, IOException {
11         Properties properties = new Properties();
12         properties.setProperty("parametrol", "valor_dos");
13         properties.store(new FileWriter("config2.properties"), "Comentario agregado");
14         properties.load(new FileReader("config2.properties"));
15         Enumeration<Object> enumeration = properties.keys();
16         while(enumeration.hasMoreElements()) {
17             System.out.println(properties.getProperty(String.valueOf(enumeration.nextElement())));
18         }
19     }
20 }
```

```
config2.properties

1 #Comentario agregado
2 #Sat Mar 24 21:04:14 ART 2018
3 parametrol=valor_dos
4
```

```
<terminated> UsoProperties [Java Application] C:\Program Files (x86)\Java\jre1.8.0_151\bin\java
valor_dos
```

PROPERTIES

▮ ***Propiedades del sistema:***

- ▮ No solamente existen propiedades únicamente en un archivo *.properties, el Sistema anfitrión en donde se ejecuta nuestra aplicación tiene propiedades establecidas.
- ▮ A través de las propiedades del sistema podemos saber, versión de la JVM, Sistema Operativo, ruta en donde se encuentran las librerías de la JRE
- ▮ Además de las ya mencionas, el desarrollador puede agregar propiedades a nivel de Sistema que pueden ser utilizadas durante todo el ciclo de vida de la aplicación.

PROPERTIES

▮ *Clase Properties - Acceso:*

▮ Se presentan tres maneras diferentes de acceder a un archivo ****.properties***

▮ ***Archivo properties en disco:*** para este caso se accede al archivo indicando la ruta de sistema del mismo, bien con respecto al lugar desde el que se esta ejecutando la aplicación o sino con una ruta absoluta

```
public void desdeDisco() throws IOException{  
    Properties prop = new Properties();  
    FileInputStream fileInputStream = new FileInputStream("archivo.properties");  
    prop.load(fileInputStream);  
    fileInputStream.close();  
}
```

PROPERTIES

▮ *Clase Properties - Acceso:*

- ▮ ***Archivo properties en el classpath de la aplicación:*** para este caso se accede al archivo, siendo una aplicación de escritorio o una aplicación web, desde el entorno de la aplicación.

```
public void desdeClasspath() throws IOException{
    Properties prop = new Properties();
    InputStream inputStream = this.getClass().
                                getClassLoader().
                                getResourceAsStream("archivo.properties");

    prop.load(inputStream);
    inputStream.close();
}
```

PROPERTIES

▮ *Clase Properties - Acceso:*

- ▮ ***Usando el contexto en una aplicación web:*** para este caso se accede al archivo obteniendo la configuración del contexto de la aplicación web.

```
public void init(ServletConfig config) throws ServletException {  
    Properties props = new Properties();  
    InputStream is = config.getServletContext().  
                        getResourceAsStream("/WEB-INF/config/recursos.properties");  
    props.load(is);  
    is.close();  
}
```